

SIO 2^{ème} année option SLAM
BLOC2 : Réalisation et maintenance de composants logiciels
GSB : Gestion des données avec SQLite sous Android

1. Maquette du projet à réaliser	2
➤ Les fenêtres de l'application	2
2. Gestion des données	3
➤ La base de données SQLite	3
➤ la classe Echantillon.....	4
➤ création de quelques enregistrements	7
3. ajout de code pour afficher les articles de la base de données dans la fenêtre Liste_Echant	8
➤ création d'un fichier de présentation des données	8
➤ Modification du code de ListeEchant.java.....	9
➤ Modification du code de la fenêtre associée à AjoutEchant.java	10
4. Annexe : exemple d'utilisation des méthodes de la classe BdAdapter	11
6. Annexe : exemple de création de plusieurs tables :	13
Bibliographie.....	14

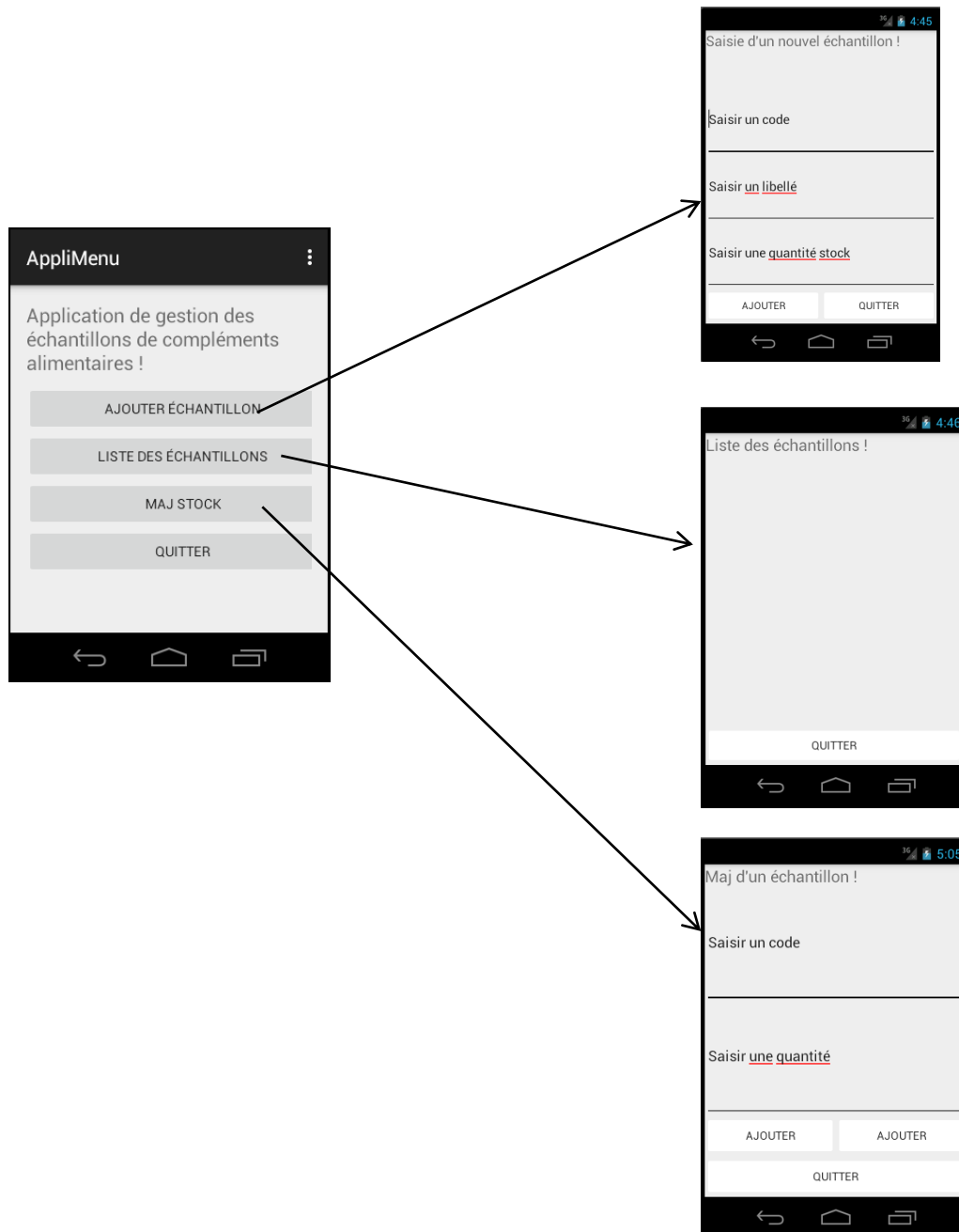
1. Maquette du projet à réaliser

Pour gérer des échantillons de compléments alimentaires, la société GSB, on dispose de la classe Echantillon.java et d'une application composée de 4 fenêtres :

- La fenêtre principale qui permet d'appeler les 3 autres fenêtres
- Une fenêtre dédiée à la saisie d'un échantillon
- Une fenêtre dédiée à l'affichage des échantillons
- Une fenêtre dédiée à la mise à jour du stock concernant un échantillon

Les données seront stockées dans dans une base de données SQLite sous Android.

➤ Les fenêtres de l'application



2. Gestion des données

Pour tester et gérer les données avec SQLite, nous allons créer un nouveau projet Android BdProjet.

➤ La base de données SQLite

SQLite est une base de données Open Source embarquée dans le système Android.

SQLite supporte les types suivants :

- ➔ **NULL** : valeur non définie
- ➔ **TEXT** : équivalent à String en Java
- ➔ **INTEGER** : équivalent à Long en Java
- ➔ **REAL** : équivalent à Double en Java
- ➔ **BLOB** : la donnée est enregistrée comme elle a été donnée

Les autres types doivent être convertis dans un de ces types définis ci-dessus avant l'insertion en base. SQLite n'offre pas de support concernant la validation de ces types lors de l'insertion.

SQLite est disponible sur n'importe quel terminal sous Android. Utiliser une base SQLite ne requiert aucune étape d'administration ou de paramétrage préalable. Il suffit simplement d'écrire les requêtes SQL permettant de créer ou de mettre à jour la base de données. Celle-ci est ensuite gérée automatiquement par la plateforme Android.

Accéder à une base de données SQLite implique en fait d'accéder à un fichier stocké quelque part sur le système de fichier. Cette opération peut être plus ou moins longue. Il est donc recommandé de réaliser toutes les opérations sur la base de données de manière asynchrone, par exemple via la classe AsyncTask de l'API Android.

Par défaut, la base de données est sauvegardée dans l'espace réservé à l'application :
/data/data/<package de l'application>/databases/<nom de la base de données>

Le sous package android.database.sqlite contient les classes spécifiques à SQLite.

La classe SQLiteDatabase est la classe principale permettant de travailler avec les bases de données SQLite. Elle offre plusieurs méthodes pour accéder à la base de données, exécuter des requêtes pour récupérer un ensemble de données ou pour mettre à jour la base et fermer la base de données :

- ➔ **insert*()** : méthodes pour insérer une donnée dans une table
- ➔ **execSQL()** : exécuter une requête SQL qui ne retourne pas ou ne modifie pas ou ne supprime pas des données
- ➔ **query*()** : retourne un Cursor sur les données en fonction de paramètres donnés
- ➔ **rawQuery()** : retourne un Cursor en prenant en entrée directement une requête SQL
- ➔ **update*()** : méthodes pour mettre à jour un ensemble de données
- ➔ **delete()** : méthode pour supprimer un ensemble de données

Les méthodes **insert*()** et **update*()** utilisent en paramètre la classe ContentValues qui décrit sous forme de clé / valeur la donnée à insérer en base.

Objet Cursor

Les méthodes **query*()** et **rawQuery()** retournent un objet Cursor. Cet objet représente le résultat de la requête exécutée et pointe généralement sur un élément de la liste du résultat. Ce principe permet d'éviter de devoir tout charger en mémoire. La classe Cursor offre les méthodes nécessaires pour naviguer dans la liste des éléments du résultat de la requête :

getCount() : retourne le nombre d'éléments du résultat de la requête

move*() : méthodes permettant de déplacer le curseur dans le résultat

isAfterLast() : permet de déterminer si on a atteint la fin de la liste du résultat

get*(int columnIndex) : ensemble de méthodes permettant de retourner la valeur typée d'une colonne donnée selon son index. Exemple : **getLong()** ou **getString()**

getColumnIndex() : retourne l'index correspondant au nom de la colonne ou retourne -1 si le nom de la colonne n'existe pas

getColumnIndexOrThrow() : retourne l'index correspondant au nom de la colonne ou lève une exception si le nom de la colonne n'existe pas

➤ **la classe Echantillon** (à créer dans le même dossier que le MainActivity.java)

Elle va nous servir pour le passage ou la récupération de paramètres depuis ou vers la base de données.

```
package com.example.applimenu;

public class Echantillon {

    protected String code;
    protected String libelle;
    protected String quantiteStock;

    public Echantillon(String code, String libelle, String qteStock) {
        this.code = code;
        this.libelle = libelle;
        this.quantiteStock = qteStock;
    }

    public Echantillon() {
        this.code = null;
        this.libelle = null;
        this.quantiteStock = null;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public void setLibelle(String libelle) {
        this.libelle = libelle;
    }

    public void setQuantiteStock(String quantiteStock) {
        this.quantiteStock = quantiteStock;
    }

    public String getCode() {
        return code;
    }

    public String getLibelle() {
        return libelle;
    }

    public String getQuantiteStock() {
        return quantiteStock;
    }
}
```

Nous allons créer une nouvelle classe **CreateBdEchantillon.java** (dans le dossier contenant le MainActivity.java) et qui hérite de **SQLiteOpenHelper**. Cette classe va nous permettre de définir la table qui sera produite lors de l'instanciation de celle-ci.

```
package com.example.applimenu;

import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class CreateBdEchantillon extends SQLiteOpenHelper {

    private static final String TABLE_ECHANT = "echantillons";
```

```

static final String COL_ID = "_id";
private static final String COL_CODE = "CODE";
private static final String COL_LIB = "LIB";
private static final String COL_STOCK = "STOCK";

private static final String CREATE_BDD = "CREATE TABLE " + TABLE_ECHANT + "
("+COL_ID+" INTEGER PRIMARY KEY AUTOINCREMENT,"
    + COL_CODE + " TEXT NOT NULL, " + COL_LIB + " TEXT NOT NULL, "
    + COL_STOCK + " TEXT NOT NULL);";

public CreateBdEchantillon(Context context, String name,
    SQLiteDatabase.CursorFactory factory, int version) {
    super(context, name, factory, version);
}

@Override
public void onCreate(SQLiteDatabase db) {
    // appelée lorsqu'aucune base n'existe et que la classe utilitaire doit
    // en créer une
    //on crée la table à partir de la requête écrite dans la variable
    CREATE_BDD
    db.execSQL(CREATE_BDD);
}

@Override // appelée si la version de la base a changé
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    //On peut supprimer la table et de la recréer
    db.execSQL("DROP TABLE " + TABLE_ECHANT + ";");
    onCreate(db);
}
}

```

Ensuite nous allons créer une nouvelle classe utilitaire **BdAdapter.java** (dans le dossier contenant le MainActivity.java). Elle va nous permettre de gérer l'insertion, la suppression, la modification des échantillons dans la Base De Données ainsi que de faire des requêtes pour récupérer un article contenu dans la base de données.

```

package com.example.applimenu;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;

public class BdAdapter {
    static final int VERSION_BDD = 1;
    private static final String NOM_BDD = "gsb.db";
    private static final String TABLE_ECHANT = "echantillons";
    static final String COL_ID = "_id";
    static final int NUM_COL_ID = 0;
    static final String COL_CODE = "CODE";
    static final int NUM_COL_CODE = 1;
    static final String COL_LIB = "LIB";
    static final int NUM_COL_LIB = 2;
    static final String COL_STOCK = "STOCK";
    static final int NUM_COL_STOCK = 3;

    private CreateBdEchantillon bdArticles;
    private Context context;
    private SQLiteDatabase db;
}

```

```

    public BdAdapter(Context context){
        this.context = context;
        bdArticles = new CreateBdEchantillon(context, NOM_BDD, null,
VERSION_BDD);
    }

    //si la base n'existe pas, l'objet SQLiteOpenHelper exécute la méthode
onCreate
    // si la version de la base a changé, la méthode onUpgrade sera lancée, dans
les 2 cas l'appel
    // à getWritableDatabase permet d'ouvrir une connexion à la base en écriture
    public BdAdapter open(){
        db = bdArticles.getWritableDatabase();
        return this;
    }
    public BdAdapter close (){
        db.close();
        return null;
    }

    public long insererEchantillon(Echantillon unEchant){
        //Création d'un ContentValues (fonctionne comme une HashMap)
        ContentValues values = new ContentValues();
        //on lui ajoute une valeur associé à une clé (qui est le nom de la
colonne où on veut mettre la valeur)
        values.put(COL_CODE, unEchant.getCode());
        values.put(COL_LIB, unEchant.getLibelle());
        values.put(COL_STOCK, unEchant.getQuantiteStock());
        //on insère l'objet dans la BDD via le ContentValues
        return db.insert(TABLE_ECHANT, null, values);
    }

    private Echantillon cursorToEchant(Cursor c){
        //Cette méthode permet de convertir un cursor en un echantillon
        //si aucun élément n'a été retourné dans la requête, on renvoie null
        Echantillon unEchant=null ;
        if (c.getCount() != 0){
            c.moveToFirst(); //on se place sur le premier élément
            unEchant = new Echantillon(); //On crée un echantillon
            //on lui affecte toutes les infos grâce aux infos contenues dans le
Cursor
            unEchant.setCode(c.getString(NUM_COL_CODE));
            unEchant.setLibelle(c.getString(NUM_COL_LIB));
            unEchant.setQuantiteStock(c.getString(NUM_COL_STOCK));
        }
        c.close(); //On ferme le cursor
        return unEchant; //On retourne l'echantillon
    }

    public Echantillon getEchantillonWithLib(String unCode){
        //Récupère dans un Cursor les valeurs correspondant à un echantillon
grâce à sa designation)
        Cursor c = db.query(TABLE_ECHANT, new String[] {COL_ID,COL_CODE,
COL_LIB,
        COL_STOCK}, COL_CODE + " LIKE \"" + unCode + "\"", null, null,
null, null);
        return cursorToEchant(c);
    }

    public int updateEchantillon(String unCode, Echantillon unEchant){
        //La mise à jour d'un echantillon dans la BDD fonctionne plus ou moins
comme une insertion,
        // il faut simple préciser quel echantillon on doit mettre à jour grâce

```

```

à son code
    ContentValues values = new ContentValues();
    values.put(COL_CODE, unEchantillon.getCode());
    values.put(COL_LIB, unEchantillon.getLibelle());
    values.put(COL_STOCK, unEchantillon.getQuantiteStock());
    return db.update(TABLE_ECHANT, values, COL_CODE + " = \"" + unCode + "\"",
null);
}

public int removeEchantillonWithCode(String unCode){
    //Suppression d'un echantillon de la BDD grâce à son code
    return db.delete(TABLE_ECHANT, COL_CODE + " = \"" + unCode + "\"", null);
}

public Cursor getData(){
    return db.rawQuery("SELECT * FROM echantillons", null);
}
}

```

➤ création de quelques enregistrements

Dans la classe **MainActivity.java** on peut rajouter le code suivant pour créer quelques enregistrements dans la base de données.

Auparavant dans la classe **BdAdapter**, changer le numero de version de la base de données pour que l'ancienne base soit écrasée.

```
static final int VERSION_BDD = 2;
```

```

public void jeuEssaiBd(){
    //Création d'une instance de la classe echantBDD
    BdAdapter echantBdd = new BdAdapter(this);

    //On ouvre la base de données pour écrire dedans
    echantBdd.open();
    //On insère DES ECHANTILLONS DANS LA BD
    echantBdd.insererEchantillon(new Echantillon("code1", "lib1", "3"));
    echantBdd.insererEchantillon(new Echantillon("code2", "lib2", "5"));
    echantBdd.insererEchantillon(new Echantillon("code3", "lib3", "7"));
    echantBdd.insererEchantillon(new Echantillon("code4", "lib4", "6"));
    Cursor unCurseur = echantBdd.getData();
    System.out.println("il y a "+String.valueOf(unCurseur.getCount())+"
echantillons dans la BD");
    echantBdd.close();
}

```

Pour tester ces 2 nouvelles classes, il suffit de rajouter la méthode suivante dans la classe **MainActivity.java** et l'appeler juste avant la gestion des boutons.

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // creation d'un jeu d'essai → ajout de quelques enregistrements
    jeuEssaiBd();
}

```

La ligne suivant apparait dans « logcat »

```

03-24 10:52:47.136 1662-1662/com.example.applimenu I/System.out: il y a 4 echantillons dans la BD

```

Une fois ces classes testées, nous allons les importer dans le projet AppliMenu du contexte GSB pour continuer ce projet.

3. ajout de code pour afficher les articles de la base de données dans la fenêtre liste_echantillon.xml

Sous Android, le composant principal pour afficher et gérer une liste de données est le composant ListView. Une ListView, comme son nom l'indique, est un Widget Android vous permettant d'afficher des éléments dans une liste avec défilement. Selon les besoins, il peut être nécessaire d'afficher plus d'informations sur chaque ligne de la liste, ou même de venir effectuer des opérations particulières suite à des actions sur les lignes.

Pour afficher les données dans un ListView, il faut utiliser un Adapter qui indiquera la présentation à utiliser dans l'HM. L'adapter permet de gérer les données et de réaliser le mapping relationnel entre les données et l'HM.

Il est aussi possible d'associer des événements sur clic dans la liste → pour modifier un article par exemple. Il va donc falloir créer le fichier XML de représentation des données dans la ListView.

➤ création d'un fichier de présentation des données

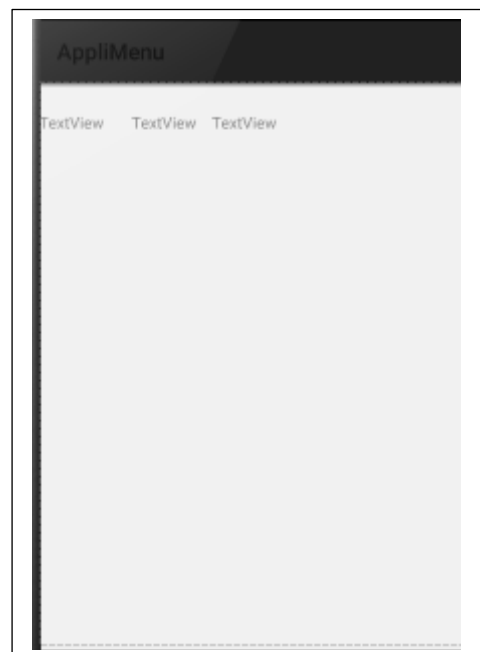
Afin de placer les enregistrements dans la ListView, il faut créer le fichier **list_entree.xml** (dans le dossier layout) :

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <TextView
        android:id="@+id/listeTextViewCode"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true"
        android:layout_marginTop="28dp"
        android:text="TextView" />

    <TextView
        android:id="@+id/listeTextViewLib"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignBaseline="@+id/listeTextViewCode"
        android:layout_alignBottom="@+id/listeTextViewCode"
        android:layout_marginLeft="25dp"
        android:layout_toRightOf="@+id/listeTextViewCode"
        android:text="TextView" />

    <TextView
        android:id="@+id/listeTextViewStock"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignBaseline="@+id/listeTextViewLib"
        android:layout_alignBottom="@+id/listeTextViewLib"
        android:layout_marginLeft="14dp"
        android:layout_marginRight="32dp"
        android:layout_toRightOf="@+id/listeTextViewLib"
        android:text="TextView" />
</RelativeLayout>
```



Dans ce fichier XML on a placé un TextView pour accueillir chacun des champs composant un echantillon.

➤ Modification du code de ListeEchant.java

Puis on crée le code de la classe **ListeEchantillon.java** afin d'accéder aux données de la base.

```
package com.example.applimenu;

import android.app.Activity;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ListView;
import android.widget.SimpleCursorAdapter;

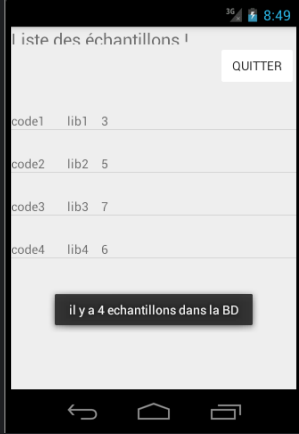
public class ListeEchantillon extends Activity {

    private ListView listViewEchant;
    private BdAdapter echantBdd;

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.liste_echantillon);
        listViewEchant = (ListView) findViewById(R.id.listeListViewEchantillon);

        echantBdd = new BdAdapter(this);
        //On ouvre la base de données pour écrire dedans
        echantBdd.open();
        Cursor leCurseur = echantBdd.getData();
        // colonnes à afficher
        String[] colNoms = new String[] {BdAdapter.COL_CODE, BdAdapter.COL_LIB,
            BdAdapter.COL_STOCK};
        // champs dans lesquelles afficher les colonnes
        int[] colNumeros = new int[] {R.id.listeTextViewCode,
R.id.listeTextViewLib, R.id.listeTextViewStock};
        // appel du fichier de présentation list_entree.xml
        SimpleCursorAdapter dataAdapter = new SimpleCursorAdapter(this,
R.layout.list_entree,
            leCurseur,colNoms,colNumeros);
        // Assign adapter to ListView
        listViewEchant.setAdapter(dataAdapter);
        echantBdd.close();

        Button buttonQuitter = (Button)findViewById(R.id.listeButtonQuitter);
        buttonQuitter.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                finish(); //fermeture de la fenêtre
            }
        });
    }
}
```



The screenshot shows the application's user interface. At the top, there's a title bar with the text 'liste des échantillons'. Below it, a list of samples is displayed with columns for 'code', 'lib', and 'stock'. The list contains four items: 'code1 lib1 3', 'code2 lib2 5', 'code3 lib3 7', and 'code4 lib4 6'. To the right of the list is a 'QUITTER' button. At the bottom of the screen, a status bar shows the time as 8:49 and the battery level at 36%. A toast message at the bottom of the screen reads 'il y a 4 échantillons dans la BD'.

N'oubliez pas d'ajouter une activité dans le **AndroidManifest.xml**

```
<activity
    android:name="AjoutEchantillon"
```

```
        android:label="@string/app_name"
    </activity>
```

>

L'étape la plus compliquée consiste à récupérer les éléments à afficher dans la base de données :

- L'objet SimpleCursorAdapter permet de créer directement la liste à partir des données contenues dans le Cursor. Il suffit de lui indiquer :
 - un Contexte (ici, notre activité),
 - un fichier de layout qui va être utilisé pour chaque élément de la liste (ici une vue contenant simplement 3 TextView dans un LinearLayout : `R.layout.list_entree`),
 - le Cursor à utiliser : `leCurseur`,
 - le nom des colonnes contenant ce que l'on veut afficher (`colNoms`),
 - puis on relie ces colonnes avec les contrôles (les TextView de `R.layout.list_entree`) où ils vont être affichés (`colNumeros`).
- La méthode `setAdapter` : permet de fournir les données formatées au ListView.

➤ **Modification du code de la fenêtre associée à `AjoutEchantillon.java`**

Ajouter maintenant le code permettant d'insérer dans la base de données l'article dont on vient de saisir les informations, puis continuez sur le projet d'application Android.

4. Annexe : exemple d'utilisation des méthodes de la classe BdAdapter

```
public void testBd(){
    //Création d'une instance de la classe echantBDD
    BdAdapter echantBdd = new BdAdapter(this);

    //Création d'un Echantillon
    Echantillon unEchant = new Echantillon("code1", "lib1", "3");

    //On ouvre la base de données pour écrire dedans
    echantBdd.open();
    //On insère l'échantillon que l'on vient de créer
    echantBdd.insererEchantillon(unEchant);
    //System.out.println("insertion échantillon");

    //Pour vérifier que l'on a bien créé un échantillon dans la BDD
    //on extrait l'échantillon de la BDD grâce au libelle de l'échantillon que l'on a créé précédemment
    Echantillon unEchantFromBdd = echantBdd.getEchantillonWithLib("lib1");
    //Si un unArticle est retourné (donc si le unEchant à bien été ajouté à la BDD)
    if(unEchantFromBdd != null){
        //On affiche les infos de l'échantillon dans un Toast
        Toast.makeText(this, unEchantFromBdd.getLibelle(), Toast.LENGTH_LONG).show();
        System.out.println("échantillon trouve : "+unEchantFromBdd.getLibelle());
        //On modifie le titre de l'échantillon
        unEchantFromBdd.setLibelle("lib2");
        //Puis on met à jour la BDD
        echantBdd.updateEchantillon(unEchantFromBdd.getCode(), unEchantFromBdd);
    }
    else {
        Toast.makeText(this, "échantillon non trouvé", Toast.LENGTH_LONG).show();
        System.out.println("échantillon non trouvé");
    }

    //On extrait l'Article de la BDD grâce à son nouveau lib
    unEchantFromBdd = echantBdd.getEchantillonWithLib("Lib2");
    //S'il existe un Article possédant cette désignation dans la BDD
    if(unEchantFromBdd != null){
        //On affiche les nouvelles info de l'échantillon pour vérifié que le lib de l'échantillon a bien été maj
        Toast.makeText(this, unEchantFromBdd.getLibelle(), Toast.LENGTH_LONG).show();
        //on supprime le unEchant de la BDD grâce à son ID
        echantBdd.removeEchantillonWithCode(unEchantFromBdd.getCode());
    }

    //On essaye d'extraire de nouveau l'échantillon de la BDD toujours grâce à son nouveau libelle
    unEchantFromBdd = echantBdd.getEchantillonWithLib("lib2");
    //Si aucun unEchant n'est retourné
    if(unEchantFromBdd == null){
        //On affiche un message indiquant que l'échantillon n'existe pas dans la BDD
        Toast.makeText(this, "Cet échantillon n'existe pas dans la BDD", Toast.LENGTH_LONG).show();
    }
    else{
        //Si l'Aechantillon existe (mais normalement il ne devrait pas)
        //on affiche un message indiquant que l'échantillon existe dans la BDD
        Toast.makeText(this, "Cet échantillon existe dans la BDD", Toast.LENGTH_LONG).show();
    }
    echantBdd.close();
}
```


6. Annexe : exemple de création de plusieurs tables :

```
public class DatabaseHelper extends SQLiteOpenHelper {
    //Initialize Variable
    private static final String DATABASE_NAME = "product_databse";
    private static final String TABLE1 = "table1";
    private static final String TABLE2 = "table2";
    private static final String TABLE3 = "table3";

    @Override
    public void onCreate(SQLiteDatabase db) {
        //Create Tables
        String table1 = "CREATE TABLE "+TABLE1+"(id INTEGER PRIMARY KEY,product TEXT)";
        String table2 = "CREATE TABLE "+TABLE2+"(id INTEGER PRIMARY KEY,qty TEXT)";
        String table3 = "CREATE TABLE "+TABLE3+"(id INTEGER PRIMARY KEY,price TEXT)";
        db.execSQL(table1);
        db.execSQL(table2);
        db.execSQL(table3);

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        //Drop Existing Table
        db.execSQL("DROP TABLE IF EXISTS "+TABLE1);
        db.execSQL("DROP TABLE IF EXISTS "+TABLE2);
        db.execSQL("DROP TABLE IF EXISTS "+TABLE3);
        onCreate(db);
    }

    //Create Insert Method
    public boolean insert(String product,String qty,String price){
        //Get WriteAble Database
        SQLiteDatabase sqLiteDatabase = this.getWritableDatabase();

        //Create ContentValues
        ContentValues values1 = new ContentValues();
        values1.put("product",product);
        //Insert Data into Database
        sqLiteDatabase.insert(TABLE1, nullColumnHack: null,values1);

        //Create ContentValues
        ContentValues values2 = new ContentValues();
        values2.put("qty",qty);
        //Insert Data into Database
        sqLiteDatabase.insert(TABLE2, nullColumnHack: null,values2);

        //Create ContentValues
        ContentValues values3 = new ContentValues();
        values3.put("price",price);
        //Insert Data into Database
        sqLiteDatabase.insert(TABLE3, nullColumnHack: null,values3);
    }
}
```

Bibliographie

sql lite

<http://www.tutomobile.fr/comment-utiliser-sqlite-sous-android-tutoriel-android-n%C2%B019/19/10/2010/>

<http://a-renouard.developpez.com/tutoriels/android/sqlite/>

<http://preprod-orange-tunisie.makina-corpus.net/developper-ensemble/tutoriel/android-introduction-sqlite>

transférer les données d'un curseur dans un ListView

<http://mickael-lt.developpez.com/tutoriels/android/personnaliser-listview/>

<http://thinkandroid.wordpress.com/2010/01/09/simplecursoradapters-and-listviews/>

<http://www.mysamplecode.com/2012/07/android-listview-cursoradapter-sqlite.html>

<http://blog.dreeckan.net/2011/10/11/utilisation-des-listview-dans-une-application-android/>

liste des différents types d'Adapter

<http://developer.android.com/reference/android/widget/Adapter.html>

<http://manu.kegtux.org/Java/Tutoriels/Android/androidAvance.html#selection>

fichiers XML

<http://thibault-koprowski.fr/2010/10/15/tutoriel-parsing-xml-sous-android/>

<http://java.developpez.com/faq/xml/?page=sax>

<http://android-france.fr/2009/06/26/ecrire-et-lire-dans-un-fichier-avec-android/>